Entregable: Ítem 1 - Cinemática Directa Verificada:

1. Fundamento Matematico y Asignacion de Marcos (Frames):

El análisis de la cadena cinemática del JetCobot se fundamentó en la asignación de marcos de referencia (frames) siguiendo el criterio Denavit-Hartenberg (DH). El *Frame 0* se ubicó en la base del robot, alineando cada eje Z posterior con el eje de giro de su respectiva articulación y utilizando el eje X como normal común.

2. Tabla Denavit-Hartenberg (DH) Resultante

Tras medir e identificar los desplazamientos físicos, se construyó la tabla DH bajo la convención estándar $[\theta, d, a, \alpha]$. Para garantizar una predicción teórica confiable con un margen de error menor a 10 mm, se emplearon los desplazamientos más exactos y se aplicaron offsets articulares estratégicos en q_2, q_4 y q_5 para alinear el cero matemático con el cero físico del motor.

TABLA DENAVIT-HARTENBERG				
Articulación	θ (rad)	d (mm)	a (mm)	α
1	q_1	132.5	0	$90°$
2	q_2° -90°	0	-110	0
3	q_3	0	-95	0
4	$q_4 - 90°$	65.5	0	90°
5	$q_5 + 90°$	74.0	0	-90°
6	q_6	49.5	0	0

Código para MatLab

```
clear; clc;
poses = struct();
poses.cero   = [0.0, 0.0, 0.0, 0.0, 0.0, 0.0];
poses.ready  = [0.0, -0.5, 0.5, 0.0, 0.5, 0.0];
poses.girada = [0.6, -0.4, 0.4, 0.0, 0.3, 0.0]
poses.baja   = [0.0, -1.2, 1.2, 0.0, 0.0, 0.0];
nombres = fieldnames(poses);
disp('--- Predicciones de Cinemática Directa (x, y, z) en mm ---');

for i = 1:length(nombres)
   nombre = nombres{i};
   q = poses.(nombre);
   posicion = calcular_fk_numerico(q);

   fprintf('Pose "%-6s" -> X: %7.2f | Y: %7.2f | Z: %7.2f\n', ...
           nombre, posicion(1), posicion(2), posicion(3));
end
function pos = calcular_fk_numerico(q)
   DH = [
       q(1),           132.5,  0.0,    pi/2;
       q(2) - pi/2,    0.0,    -110.0,  0.0;
       q(3),           0.0,    -95.0,   0.0;
       q(4) - pi/2,    65.5,    0.0,    pi/2;
       q(5) + pi/2,    74.0,   0.0,   -pi/2;
       q(6),           49.5,    0.0,    0.0
   ];
   T_total = eye(4);
   for i = 1:6
       th = DH(i,1);
       d  = DH(i,2);
       a  = DH(i,3);
       alf= DH(i,4);
       T_i = [
           cos(th), -sin(th)*cos(alf),  sin(th)*sin(alf), a*cos(th);
           sin(th),  cos(th)*cos(alf), -cos(th)*sin(alf), a*sin(th);
           0.0,      sin(alf),          cos(alf),         d;
           0.0,      0.0,               0.0,              1.0
       ];

       T_total = T_total * T_i;
   end
   pos = T_total(1:3, 4);
end
```

```
--- Predicciones de Cinemática Directa (x, y, z) en mm ---
Pose "cero  " -> X:   49.50 | Y:  -65.50 | Z:  411.50
Pose "ready " -> X:   96.18 | Y:  -41.77 | Z:  398.03
Pose "girada" -> X:  103.11 | Y:    8.90 | Z:  402.82
Pose "baja  " -> X:  152.02 | Y:  -65.50 | Z:  341.36
```

Resultado Matlab

3. Implementación en Código

Dicha tabla se incorporó en el script fk.py. A través de la función fk(q), se ingresa el vector articular de los 6 grados de libertad para calcular de forma

secuencial las 6 matrices de transformación, obteniendo como resultado las coordenadas espaciales finales (x, y, z) correspondientes al efector final.

```
import math

JOINT_NAMES = ['1_Joint', '2_Joint', '3_Joint', '4_Joint', '5_Joint',
'6_Joint']

# ============================== IMPLEMENTAR · ítem 1 ==================
# Tabla DH del JetCobot: una fila (alpha_rad, a_mm, d_mm, theta_offset_rad)
por
# articulación, en convención DH estándar. Dedúzcanla y verifíquenla con
# herramientas/verificar_fk.py contra el robot: criterio ≤ 10 mm.

DH = [
    (math.pi / 2.0,   0.0,    132.5,    0.0),
    (0.0,            -111.5,  0.0,     -math.pi / 2.0),
    (0.0,            -95.0,   0.0,      0.0),
    (math.pi / 2.0,   0.0,    65.5,    -math.pi / 2.0),
    (-math.pi / 2.0,  0.0,    74.0,     math.pi / 2.0),
    (0.0,             0.0,    49.5,     0.0),
]

JOINT_LIMITS = [
    (-2.93, 2.93),
    (-2.36, 2.36),
    (-2.53, 2.53),
    (-2.58, 2.58),
    (-2.93, 2.93),
    (-3.14, 3.14),
]

ALCANCE_MIN_MM = 80.0
ALCANCE_MAX_MM = 480.0


def _t(alpha, a, d, theta):
    ca, sa = math.cos(alpha), math.sin(alpha)
    ct, st = math.cos(theta), math.sin(theta)
    return [
        [ct,  -st * ca,   st * sa,  a * ct],
        [st,   ct * ca,  -ct * sa,  a * st],
        [0.0,  sa,        ca,       d],
        [0.0,  0.0,       0.0,      1.0],
    ]


def _mul(A, B):
    return [[sum(A[i][k] * B[k][j] for k in range(4)) for j in range(4)] for i
in range(4)]


def fk_matriz(q):
    if len(q) != 6:
        raise ValueError(f'se esperaban 6 ángulos, llegaron {len(q)}')
    T = [[1.0 if i == j else 0.0 for j in range(4)] for i in range(4)]
    for (alpha, a, d, off), theta in zip(DH, q):
        T = _mul(T, _t(alpha, a, d, theta + off))
    return T


def fk(q):
    T = fk_matriz(q)
```

```
    x = T[0][3]
    y = T[1][3]
    z = T[2][3]
    return x, y, z

# ====================================================================

def dentro_de_limites(q):
    if len(q) != 6:
        return False, f'se esperaban 6 ángulos, llegaron {len(q)}'
    for i, (valor, (lo, hi)) in enumerate(zip(q, JOINT_LIMITS)):
        if not lo <= valor <= hi:
            return False, f'{JOINT_NAMES[i]} fuera de rango: {valor:.3f} rad,
límite [{lo}, {hi}]'
    return True, ''

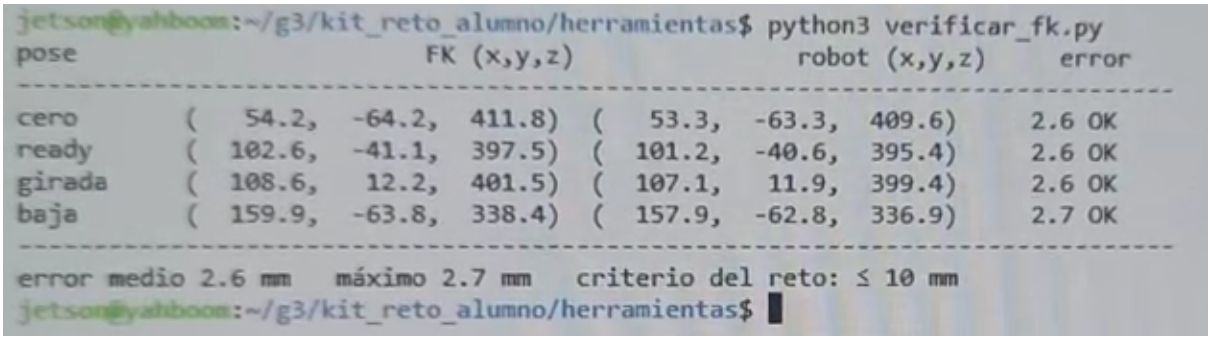
def dentro_del_workspace(q):
    x, y, z = fk(q)
    r = math.sqrt(x * x + y * y + z * z)
    if r > ALCANCE_MAX_MM:
        return False, f'efector a {r:.0f} mm de la base, máximo
{ALCANCE_MAX_MM:.0f}'
    if r < ALCANCE_MIN_MM:
        return False, f'efector a {r:.0f} mm de la base, demasiado cerca'
    if z < 0.0:
        return False, f'z = {z:.0f} mm: el efector quedaría bajo la base'
    return True, ''

def paso_articular(q_desde, q_hasta):
    return max(abs(b - a) for a, b in zip(q_desde, q_hasta))
```

4. Validación Física y Registro del Error Cartesiano

Para llevar a cabo la verificación, se corrió el programa verificar_fk.py en la unidad Jetson tras habilitar el puerto de comunicación. El sistema envió las órdenes necesarias para posicionar el manipulador en las configuraciones de evaluación *"cero"*, *"ready"* y *"baja"*, contrastando de forma directa las lecturas obtenidas con la estimación del modelo teórico.

Captura de Evidencia:



```
jetson@yahboom:~/g3/kit_reto_alumno/herramientas$ python3 verificar_fk.py
pose                    FK (x,y,z)                robot (x,y,z)     error
------------------------------------------------------------------------
cero        (   54.2,  -64.2,  411.8)  (   53.3,  -63.3,  409.6)    2.6 OK
ready       (  102.6,  -41.1,  397.5)  (  101.2,  -40.6,  395.4)    2.6 OK
girada      (  108.6,   12.2,  401.5)  (  107.1,   11.9,  399.4)    2.6 OK
baja        (  159.9,  -63.8,  338.4)  (  157.9,  -62.8,  336.9)    2.7 OK
------------------------------------------------------------------------
error medio 2.6 mm   máximo 2.7 mm   criterio del reto: ≤ 10 mm
jetson@yahboom:~/g3/kit_reto_alumno/herramientas$
```